Wooyoung Kim
323-834-4126
wkim9450@usc.edu

EE 569 Homework #6
April 28, 2026

I. Answers to the Non-programming Questions

I..1 Feedforward-designed Convolutional Neural Networks

(1) Summary of the Saab Transform

The Saab transform is a deterministic alternative to the non-linear activation functions(ex. ReLU) used in traditional CNNs. In standard PCA, cascaded layers face a sign problem where negative projections can cancel out structural information. The Saab transform solves this by extracting the AC components of a data patch, performing PCA, and then adding a strictly computed, data driven constant bias. This bias ensures all output features are non-negative, allowing hierarchical feature extraction without needing backpropagation.

Flow:

1. **Patch Extraction:** Extract local overlapping patches from the input image or previous feature map.
2. **DC Removal:** Calculate and subtract the local mean(DC component) from each patch to isolate the spatial variations(AC components).
3. **PCA:** Apply PCA on the AC components to find orthogonal basis vectors.
4. **Bias Adjustment:** Add a maximum bias value (derived from the minimum projection values of the training data) to the AC projections to guarantee all outputs are non-negative.
5. **Concatenation:** Combine the DC component and the bias-adjusted AC components to form the final output features for the next layer.

(2) Similarities and Differences between FF-CNN and BP-CNN

Similarities:

- Both use a cascaded, hierarchical architecture to extract features from coarse spatial levels to fine semantic levels.
- Both utilize spatial pooling (ex. max pooling) to reduce dimensionality and increase the receptive field.
- Both have two distinct phases: feature extraction (convolutional layers) and classification mapping (fully-connected layers).

Differences:

- BP-CNNs learn weights iteratively through gradient descent and backpropagation by minimizing a loss function. FF-CNNs calculate weights mathematically in a single forward pass using subspace learning(PCA/SVD).
- BP-CNNs rely on non-linear activation functions (ReLU, Sigmoid). FF-CNNs use the Saab transform's adjusted bias to handle non-negativity.

Advantages of FF-CNN:

- Highly interpretable due to mathematical transparency.
- Fast training time since there are no epochs or iterative gradient updates.
- Requires less training data to converge and is less prone to overfitting.

Limitations of FF-CNN:

- May not achieve the peak accuracies of massive, heavily optimized BP-CNNs on very complex, large-scale datasets.
- Less flexible for arbitrary end-to-end mapping tasks outside of standard classification, whereas deep learning is highly versatile.

I..2 Understanding PixelHop and PixelHop++

(1) SSL Methodology and Comparison with DL

SSL is a feature extraction methodology that progressively breaks down high-dimensional input data into smaller, meaningful subspaces. At each stage, it analyzes the statistical distribution of the data, identifies the principal components holding the most structural energy, and discards low energy noise.

DL vs. SSL Comparison:

- **Philosophy:** Deep Learning is an black box methodology that searches for parameters through iterative trial and error. SSL is a statistically driven white box methodology that computes parameters analytically based on data variance.
- **Training Efficiency:** Deep Learning requires extensive computational resources and long training times over multiple epochs. SSL trains sequentially layer by layer in a single pass, making it highly efficient and often executable on standard CPUs.

(2) Functions of Modules 1, 2, and 3

- **Module 1 (Neighborhood Construction):** Extracts local patches from the input to capture spatial context and structural relationships between neighboring pixels or features.

- **Module 2 (Subspace Approximation):** Applies linear transformations to the constructed neighborhoods to reduce dimensionality, extracting the most energy-dense, orthogonal feature representations.
- **Module 3 (Information Allocation & Label Assignment):** Evaluates the extracted features, typically splitting them into relevant subsets(kept for the next stage) and irrelevant subsets. It then uses statistical models (like Linear Least Squares Regression or Random Forests) to map the pooled features to final classification labels.

(3) Neighborhood Construction, Subspace Approximation, Saab vs c/w Saab

PixelHop vs. PixelHop++:

- **PixelHop:** In its neighborhood construction, it concatenates features from all input channels simultaneously. Its subspace approximation computes a single, massive global covariance matrix across the combined spatial and spectral dimensions.
- **PixelHop++:** Constructs neighborhoods for each channel (or small subsets of channels) independently. Its subspace approximation processes these channels separately, which cuts down mathematical complexity.

Basic Saab vs. Channel-wise (c/w) Saab Transform:

- **Basic Saab Transform:** Applies PCA to a joint tensor containing both spatial pixels and all input channels. As network depth and channel count increase, the covariance matrix size grows exponentially, leading to severe memory and computational bottlenecks.
- **Channel-wise Saab Transform:** Applies PCA separately to each individual channel. By breaking a massive multi channel covariance matrix into many small, single channel matrices, the c/w Saab transform significantly reduces model size, memory footprint, and training time, while preserving critical feature information.

II. Abstract and Motivation

Convolutional Neural Networks have dominated image classification but are often criticized for being black boxes with high computational costs. The motivation of this section is to implement and analyze the Saab Transform and its evolution, PixelHop++, as a mathematically transparent and efficient alternative. We aim to demonstrate that PixelHop++ can achieve comparable accuracy to standard PixelHop while reducing the number of model parameters by decoupling spatial and channel correlations.

III. Approach and Procedures

III..1 Building PixelHop++ Model

- **Data Preprocessing:** Both MNIST and Fashion-MNIST images were zero-padded from 28×28 to 32×32 to facilitate neighborhood construction without spatial degradation. I utilized a balanced subset of 10000 training images (1000 per class).

- **Feature Extraction:** Three PixelHop++ units were utilized. Each unit consists of neighborhood construction (5×5 window, 1 stride) followed by a Channel-wise Saab Transform.
- **Max-Pooling:** Between Hop-1/Hop-2 and Hop-2/Hop-3, 2×2 max-pooling was applied to down-sample spatial dimensions.
- **Classification:** Features from the final Hop-3 stage were flattened and fed into an XGBoost classifier configured with 100 estimators.
- **Thresholds:** Energy thresholds for intermediate nodes ($TH1$) and discarded nodes ($TH2$) were set to 0.005 and 0.001.

III..2 Comparison between PixelHop and PixelHop++

To establish a baseline, I compared the PixelHop with PixelHop++ using identical configurations. The primary difference is that standard Saab calculates kernels across the entire input depth, resulting in dense feature matrices, while PixelHop++ processes each channel independently.

III..3 Error Analysis

Error analysis was conducted using the full training set of 60000 images for Fashion-MNIST and MNIST. I computed confusion matrices to visualize classification overlaps and identified the classes with the highest and lowest error rates to determine confusing class groups.

IV. Experimental Results

IV..1 Building PixelHop++ Model

Table 1 shows the performance of the PixelHop++ model ($TH1 = 0.005$) trained on 10000 images.

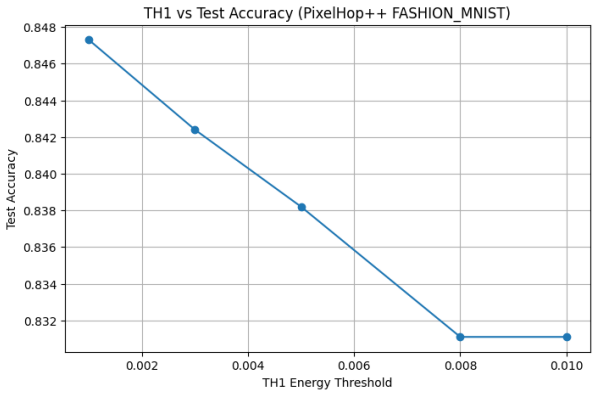
Table 1: PixelHop++ Baseline Performance ($TH1 = 0.005$)

Dataset	Training Time	Train Acc	Test Acc	Total Parameters
Fashion-MNIST	115.52	100.00%	83.82%	3650
MNIST	173.86	100.00%	95.61%	6350

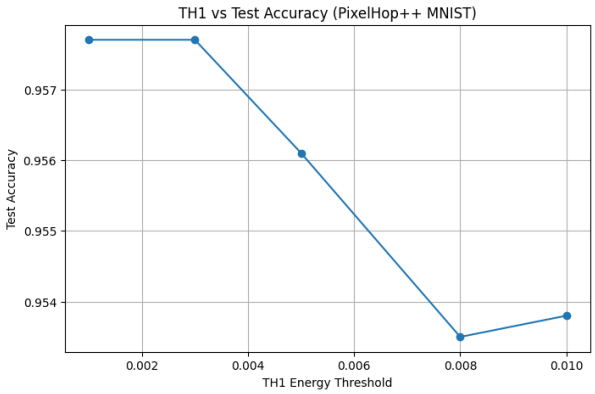
TH1 Sensitivity: I also varied $TH1$ to observe its impact on test accuracy and model size, keeping $TH2$ constant at 0.001. The results are shown in Table 2, and the corresponding accuracy curves are plotted in Figure 1.

Table 2: Effect of TH1 on Test Accuracy and Model Size

	Fashion-MNIST		**MNIST**	
TH1 Threshold	**Test Acc(%)**	**Params**	**Test Acc(%)**	**Params**
0.001	84.73	3950	95.77	6450
0.003	84.24	3850	95.77	6450
0.005	83.82	3650	95.61	6350
0.008	83.11	3450	95.35	6000
0.010	83.11	3300	95.38	5775



(a) Fashion-MNIST TH1 Curve



(b) MNIST TH1 Curve

Figure 1: Impact of TH1 Energy Threshold on Test Accuracy

IV..2 Comparison between PixelHop and PixelHop++

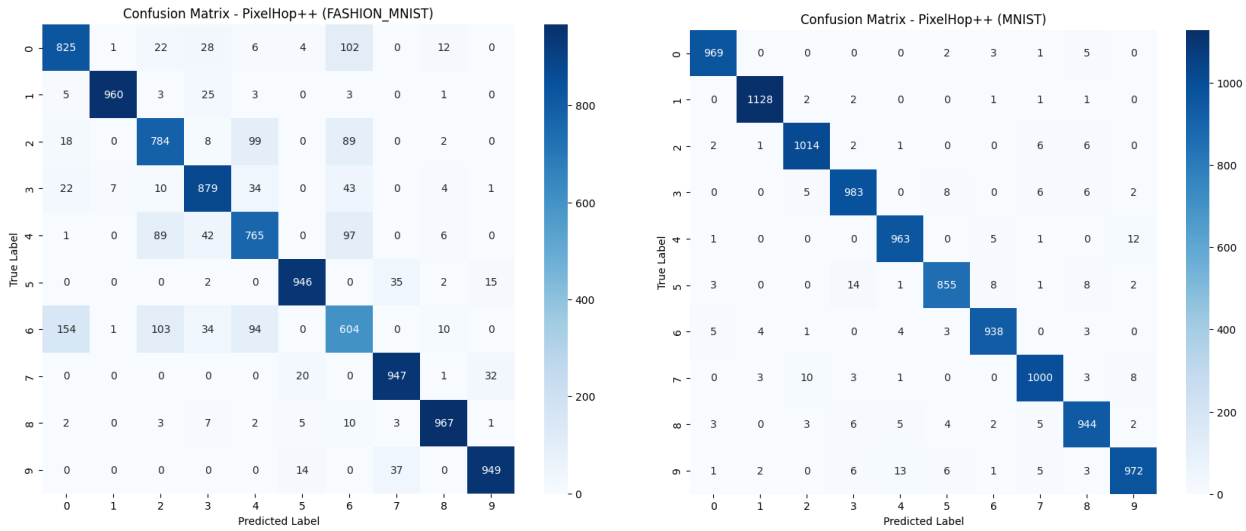
Table 3 shows the difference in computational summarization between the two architectures.

Table 3: Standard vs. Channel-wise

Dataset / Model	Train Acc	Test Acc	Runtime	Total Parameters
Fashion-MNIST				
PixelHop	100.00%	84.88%	55.07s	68750
PixelHop++	100.00%	83.82%	133.71s	3650
MNIST				
PixelHop	100.00%	95.57%	57.82s	120750
PixelHop++	100.00%	95.61%	200.62s	6350

IV..3 Error Analysis

Figure 2 presents the heat maps of the confusion matrices generated using 60000 training images.



(a) Fashion-MNIST Confusion Matrix

(b) MNIST Confusion Matrix

Figure 2: Confusion Matrices for Error Analysis

Based on the row/column totals, the extremes are:

- **Fashion-MNIST:** Lowest Error: Class 8 (Bag, 0.0330), Highest Error: Class 6 (Shirt, 0.3960).
- **MNIST:** Lowest Error: Class 1 (Digit 1, 0.0062), Highest Error: Class 5 (Digit 5, 0.0415).

V. Discussion

V..1 Building PixelHop++ Model

The $TH1$ threshold dictates how many nodes are retained based on their statistical energy (variance). As seen in Table 2 and Figure 1, increasing $TH1$ strictly decreases accuracy because more high frequency spatial information is discarded. However, it also reduces the model size. A threshold of 0.005 provides an optimal balance, stripping away noise while maintaining critical edge features without expanding the parameter count.

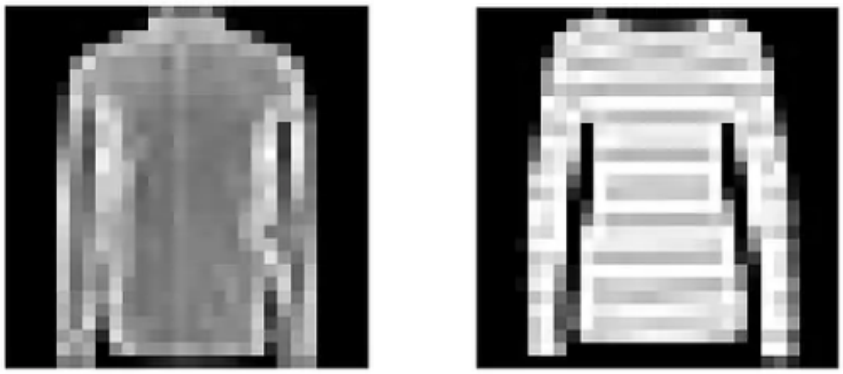
V..2 Comparison between PixelHop and PixelHop++

The core advantage of PixelHop++ is parameter efficiency. On MNIST, PixelHop++ achieves a slightly better accuracy while using approximately 19 times fewer parameters.

Standard Saab suffers from parameter inflation because deeper hops require 3D spatial spectral filtering across all incoming channels simultaneously. The c/w Saab transform circumvents this, proving that decoupled spatial filtering is highly effective. The increased runtime is an artifact of the sequential loop iteration over channels in the Python implementation, rather than a theoretical mathematical bottleneck.

V..3 Error Analysis

Confusing Class Groups: In Fashion-MNIST, Class 6(Shirt) is heavily confused with Class 0(T-shirt/top) and Class 2(Pullover). In MNIST, Class 5 is sometimes confused with Class 3.



(a) Class 6 (Shirt) (b) Class 2 (Pullover)

Figure 3: Exemplary Images of Confusing Classes

Reason of Confusion As seen in Figure 3, Shirts and Pullovers share identical structures (a central body with two sleeves). Because PixelHop++ relies heavily on max pooling down, the fine-grained, high-frequency textural differences (ex. the presence of buttons or a specific collar type) are filtered out, leaving only the dominant silhouette shape, which is nearly identical for both classes.

Improvement Proposal: To improve the classification of these classes without implementing a deep CNN, I propose utilizing Spatial Ensemble PixelHop. Instead of strictly using 5×5 windows, we could run a parallel PixelHop cascade using smaller 3×3 windows with a lower $TH1$ threshold specifically tuned to capture fine textures (ex. buttons, collars). The features from both the large shape cascade (5×5) and the fine-texture cascade (3×3) would then be concatenated before being fed into XGBoost. This would allow the classifier to distinguish between items with the same silhouette but different details.